

HOME CREDIT DEFAULT RISK — ROUND 1 SUBMISSION

# Credit Risk Intelligence Platform

*An end-to-end system that scores a loan applicant, explains why in plain English, states the policy rules behind that score, and answers free-form questions about the underlying data — all from one deployed application.*

DATASET

**Home Credit (307,511 applicants)**

MODEL

**LightGBM, ROC-AUC 0.7695**

STACK

**FastAPI + React + Postgres**

AUTHOR

**Jash Kevadiya**

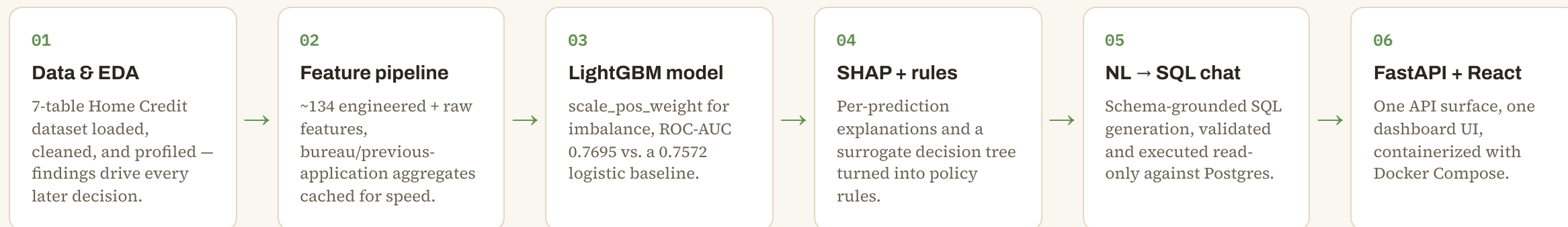
# Why credit default prediction matters

- 01 A bank approves loans without knowing, upfront, which applicants will struggle to repay — every default is a direct financial loss.
- 02 A blanket accuracy metric is misleading here: with only ~8% of applicants defaulting, a model that always predicts "no default" looks 92% accurate while being useless.
- 03 Risk officers need a \*reason\*, not just a score — a rejected applicant, and any regulator, can ask "why," and "the model said so" is not an acceptable answer.
- 04 Policy teams think in rules ("reject if X and Y"), not probabilities — the model's judgment needs to be translated into that language to be actionable.
- 05 Analysts who are not SQL-fluent still need to interrogate the data directly — "how many applicants with bad bureau debt also defaulted?" shouldn't require an engineer.
- 06 All of this has to be one coherent, deployed system — a notebook full of separate scripts is not something a bank could actually put in front of a loan officer.

8.07%

of applicants in the training data had real payment difficulty — the central class-imbalance problem this platform is built around

# Architecture overview



# What the data actually looks like

307,511

applicants

122

raw columns

8.07%

default rate

67

cols w/ missing

## Six findings that shaped the build

- 1. Most missing data is optional apartment/building detail — not a data error.
- 2. External credit scores (EXT\_SOURCE 1/2/3) are the strongest single predictor of default.
- 3. Younger applicants default roughly 2x more than the 56–65 group.
- 4. Unemployed/maternity-leave and lower-education segments run well above average risk.
- 5. The DAYS\_EMPLOYED "365243" placeholder is itself a useful, lower-risk retiree signal.
- 6. Zero bureau credit history is a mild risk signal — "unknown" isn't the same as "safe".

### Credit Risk Intelligence Platform

Home Credit Default Risk: exploratory analysis, risk scoring, explainability, policy rules, and a talk to data assistant.

#### EDA

Explore the data

#### Risk Prediction

Score and explain an applicant

#### Decision Rules

The model's policy

#### Chatbot

Ask in plain English

Built end to end: data pipeline, ML model, explainability, business rules, and a talk to data chatbot, all working on the real dataset.

### Exploratory Data Analysis

A first look at who is applying for these loans, what shape the data is in, and the patterns that matter most for spotting risk. Computed once in the analysis notebook and shown here as is.

**How this was built (in plain terms):** We loaded every loan application into a table, checked which pieces of information were missing or unusual, and drew charts to spot patterns before building any prediction model. Nothing here uses AI, it is careful counting and charting of the real data.

307,511

Applicants

122

Features

8.07%

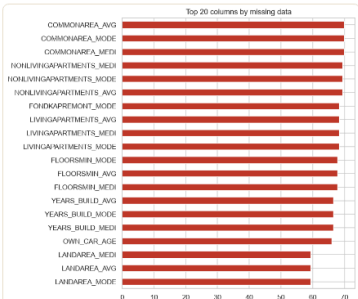
Had payment trouble

67

Columns with missing data

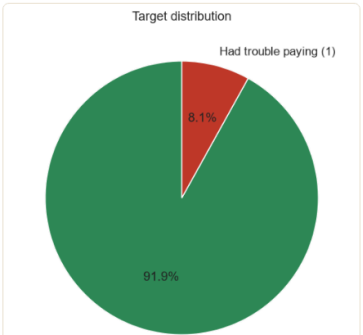
#### Most missing data is in optional building/apartment fields

The top missing-value columns are almost entirely apartment/building details. Expected, since many applicants rent or live with family and simply have nothing to report there, not a data error.



#### Only ~8% of applicants had trouble paying

A real class imbalance. A lazy model could predict 'no trouble' every time and still look 92% accurate while being useless. Drives the class-imbalance handling in the model.



# Machine learning model

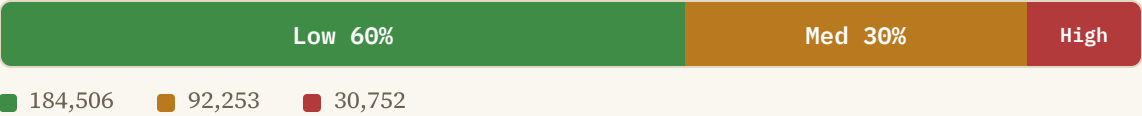
| MODEL                          | ROC-AUC |
|--------------------------------|---------|
| Logistic Regression (baseline) | 0.7572  |
| LightGBM (deployed)            | 0.7695  |

| METRIC                  | VALUE  |
|-------------------------|--------|
| PR-AUC                  | 0.2557 |
| KS statistic            | 0.402  |
| F1 @ threshold 0.67     | 0.3169 |
| Recall @ threshold 0.67 | 40.6%  |

**Class imbalance handling**

scale\_pos\_weight = 11.39 (= non-defaulters / defaulters), not SMOTE or undersampling — reweights the loss instead of synthesizing or discarding real rows.

**Risk bands (percentile cutoffs, not fixed values)**



**Why LightGBM over XGBoost**

Both now handle categorical features and histogram-based splitting natively. LightGBM does both out of the box for this dataset's mix of categorical and high-cardinality columns with no extra configuration — a convenience call, not a capability gap.

# Every score comes with a reason

**Applicant #100002 (real example)**

Score **848 / 1000** **High risk**

Actual outcome on record: had payment trouble — the model's High call was correct for this applicant.

**Why only the top 5 factors?**

SHAP computes a contribution for all ~134 features internally, but showing every one to a non-technical reader defeats the purpose of "explainable." The top 5 is what a credit officer can actually act on.

**Credit Risk Intelligence Platform**  
Home Credit Default Risk: exploratory analysis, risk scoring, explainability, policy rules, and a talk to data assistant.

EDA  
Explore the data

**Risk Prediction**  
Score and explain an applicant

Decision Rules  
The model's policy

Chatbot  
Ask in plain English

Built end to end: data pipeline, ML model, explainability, business rules, and a talk to data chatbot, all working on the real dataset.

**Risk Prediction and Explainability**  
Enter an applicant's ID to get a default probability, a 0-1000 risk score, a Low, Medium or High risk band, and a plain English breakdown of what drove that result. Try one of the real applicants below, or type your own ID.

**How this was built (in plain terms):** A machine learning model studied thousands of past loan applications where we already know what happened, meaning who repaid and who struggled, and learned the patterns that separated the two groups. For every guess it makes, a second technique called SHAP works backward through that same calculation to show which pieces of information pushed the guess up, which pushed it down, and roughly by how much, so the result is never just a mystery number.

**Example applicants**  
100002 100003 451071 429655 166551 130914 150369 437201 298124 290903

**Analyze**

**Risk score**

| Applicant | Risk score | Default probability |
|-----------|------------|---------------------|
| #100002   | 848 / 1000 | 84.8%               |

Risk band

High

This is a real applicant from the training data. Their actual recorded outcome was **had payment trouble**.

**Top drivers**

external credit score #3 (0.14) increased the risk score

external credit score #1 (0.08) increased the risk score

external credit score #2 (0.26) increased the risk score

applicant's age (25.90) decreased the risk score

credit term (0.06) increased the risk score

Red bars pushed the risk score up, green bars pulled it down. Bar length shows how much each factor mattered relative to the others shown.

Explainability

06 / 10

# Turning the model into policy rules

| DEFAULT RATE | LIFT  | SUPPORT |
|--------------|-------|---------|
| 33.6%        | 4.16× | 4,378   |
| 22.8%        | 2.82× | 9,076   |
| 21.7%        | 2.69× | 8,869   |
| 15.1%        | 1.87× | 16,382  |
| 2.1%         | 0.25× | 30,501  |

**Honest limitation:** 13 of 16 rules split only on the two external bureau scores (EXT\_SOURCE\_2/3) — they dominate the model's decision surface. Confirmed independently by both EDA and SHAP, not a rule-miner limitation.

Credit Risk Intelligence Platform

Home Credit Default Risk: exploratory analysis, risk scoring, explainability, policy rules, and a talk to data assistant.

EDA

Explore the data

Risk Prediction

Score and explain an applicant

Decision Rules

The model's policy

Chatbot

Ask in plain English

Built end to end: data pipeline, ML model, explainability, business rules, and a talk to data chatbot, all working on the real dataset.

Decision Rules

The model's overall logic, distilled into a small set of plain "if this, then that" statements. A static summary a credit policy team could review on its own, without needing to run any prediction. Sorted from highest to lowest observed risk.

How this was built (in plain terms):

We asked a much simpler, easy to read version of the model to copy the behaviour of the real, more complicated model. That simpler version naturally organises itself into a short list of plain rules, which is what you see below. Each rule also shows how many real applicants fall into it and what actually happened to them, so the rules can be checked against reality, not just trusted blindly.

IF external credit score #3 <= 0.48 AND external credit score #2 <= 0.47 AND external credit score #3 <= 0.27 AND external credit score #2 <= 0.21 THEN observed default rate is 33.6% (4,378 applicants, 4.16x the average rate)

IF external credit score #3 <= 0.48 AND external credit score #2 <= 0.47 AND external credit score #3 > 0.27 AND external credit score #2 <= 0.16 THEN observed default rate is 22.8% (9,076 applicants, 2.82x the average rate)

IF external credit score #3 <= 0.48 AND external credit score #2 > 0.47 AND external credit score #3 <= 0.27 AND external credit score #2 > 0.21 THEN observed default rate is 21.7% (8,869 applicants, 2.69x the average rate)

IF external credit score #3 <= 0.48 AND external credit score #2 > 0.47 AND external credit score #3 <= 0.27 AND external credit score #1 <= 0.58 THEN observed default rate is 15.1% (16,382 applicants, 1.87x the average rate)

IF external credit score #3 > 0.48 AND external credit score #2 <= 0.50 AND years at current job <= 8.07 AND external credit score #2 <= 0.23 THEN observed default rate is 13.0% (7,177 applicants, 1.61x the average rate)

IF external credit score #3 <= 0.48 AND external credit score #2 <= 0.47 AND external credit score #3 > 0.27 AND external credit score #2 > 0.16 THEN observed default rate is 12.7% (36,376 applicants, 1.57x the average rate)

Rule Derivation

07 / 10

# Natural language, real SQL, grounded answers

Credit Risk Intelligence Platform

Home Credit Default Risk: exploratory analysis, risk scoring, explainability, policy rules, and a talk to data assistant.

EDA

Explore the data

Risk Prediction

Score and explain an applicant

Decision Rules

The model's policy

Chatbot

Ask in plain English

Challenges and more (e.g. the 10 highest earners who had payment difficulty)

4. Ratios between two groups (e.g. loan to income ratio, defaulted vs not)

5. Questions spanning multiple tables (e.g. applicants with bad debt on their credit bureau record)

How this stays fast and cheap to run (token optimization):

- The AI only ever sees a compact list of table and column names, not long descriptions of every field, so each request stays small.
- Query results are capped at 200 rows by the database, and only the first 50 are ever shown to the step that writes the final answer, so one huge result never balloons a request.
- Follow-up questions are first condensed into a single, self-contained question by a small dedicated step, instead of replaying the entire conversation into every request as the chat grows longer.

What is the default rate by education level?

Lower secondary: 10.93%

Secondary / secondary special: 8.94%

Incomplete higher: 8.48%

Higher education: 5.36%

Academic degree: 1.83%

▼ Show SQL and raw rows (5 rows)

SELECT name\_education\_type, AVG(target) AS default\_rate, COUNT(\*) AS n\_applicants FROM

Try asking

What is the average annual income of all applicants?

What is the default rate by education level?

Show the 10 applicants with the highest income who had payment difficulty.

What is the average loan-to-income ratio for applicants who defaulted versus those who didn't?

How many applicants have at least one bad debt record with the credit bureau?

Ask a question about the data...

Send

Built end to end: data pipeline, ML model, explainability, business rules, and a talk to data chatbot, all working on the real dataset.

## Token optimization

- Compact schema card (columns + types only), not verbose per-field docs.
- Rows capped at 200 by the DB; only 50 ever reach the answer-writer.
- Follow-ups are condensed to one standalone question by a dedicated small step, instead of replaying full history every turn.

## Hallucination mitigation

- Schema grounded in Postgres's real information\_schema + real categorical values — not memorized guesses.
- Every SQL statement actually executes read-only before anything is treated as fact.
- Answer synthesis is closed-book: only the real returned rows, never a number invented to fill a gap.
- Three defense-in-depth SQL safety layers: prompt rules, AST validator (SELECT-only, table allowlist), READ ONLY transaction.



# One command, fully containerized

```
$ docker compose up --build
✓ db      Healthy
✓ api     Started
✓ frontend Started
```

- What happens automatically**
- 1. Postgres starts and passes its healthcheck.
  - 2. API waits on that healthcheck, then idempotently ingests all 7 tables.
  - 3. Model + EDA charts are already baked into the API image — nothing retrains.
  - 4. Frontend is a static build served by nginx, API URL baked in at build time.

Credit Risk Intelligence API0.1.0OAS 3.1

default

GET/healthHealth

eda

GET/eda/summarySummary

GET/eda/insightsInsights

predict

POST/predictPredict Applicant

explain

POST/explainExplain Applicant

rules

GET/rulesGet Rules

chat

POST/chatChat

Schemas

# Results summary

## Delivered

- ✓ EDA with 8 documented business insights
- ✓ LightGBM model, class-imbalance handled, ROC-AUC 0.7695
- ✓ SHAP explainability on every prediction
- ✓ 16 rules derived from the model's own behavior
- ✓ NL-to-SQL chatbot, grounded & validated
- ✓ Multi-section dashboard UI, dockerized end to end

## Known limitations

- Rules lean heavily on the two external bureau scores.
- Metrics are out-of-fold CV estimates — no labeled held-out test set exists.
- No hyperparameter search beyond early-stopping tree count.
- Single LLM provider (OpenAI) for the chatbot, not provider-agnostic.

Full reasoning behind every design choice is in `DECISIONS.md` in the repository.